TestWeaver: Execution-aware, Feedback-driven Regression Testing Generation with Large Language Models

Cuong Chi Le
FPT Software AI Center
Hanoi, Vietnam
lechicuongldk@gmail.com

Cuong Duc Van
FPT Software AI Center
Hanoi, Vietnam
duccuongtoan@gmail.com

Tung Duy Vu
VinUniversity
Hanoi, Vietnam
vdtung21.work@gmail.com

Minh Vu Thai Pham
FSOFT AI CENTER
Hanoi, Vietnam
thaiminhpv@gmail.com

Nhat Hoang Phan
Nanyang Tech. University
Singapore, Singapore
johnphan1608@gmail.com

Huy Phan
FPT AI Residency
Hanoi, Vietnam
huypn168@gmail.com

Tien N. Nguyen
Univ. of Texas at Dallas
Richardson, TX, USA
tien.n.nguyen@utdallas.edu

Abstract

While recent advances in large language models (LLMs) have shown promise in automating test generation for regression testing, they often suffer from limited reasoning about program execution, resulting in stagnated coverage growth—a phenomenon known as the coverage plateau. This paper presents TestWeaver, a novel LLM-based approach that integrates lightweight program analysis to create a focused execution context that assists LLMs in better test generation. TestWeaver strategically chooses the following components to overcome LLMs' limited reasoning on complex execution: (1) it reduces hallucinations and improves focus by supplying the LLM with the backward slice from the target line instead of a full program context; (2) it identifies and incorporates close test cases—those that share control-flow similarities with the path to the target line—to provide focused execution context within the LLM's context window; and (3) it enhances LLM's reasoning with execution in-line annotations that encode variable states as comments along the executed path. By equipping LLMs with these targeted and contextualized inputs, it improves coverage-guided test generation and mitigates redundant explorations. Empirical results show that TestWeaver accelerates code coverage growth and generates more effective test cases than the state-of-the-art approaches.

CCS Concepts

• **Software and its engineering**;

Keywords

AI4SE, Automated Test Generation, Large Language Models

ACM Reference Format:
Cuong Chi Le, Cuong Duc Van, Tung Duy Vu, Minh Vu Thai Pham, Nhat Hoang Phan, Huy Phan, and Tien N. Nguyen. 2026. TestWeaver: Execution-aware, Feedback-driven Regression Testing Generation with Large Language Models. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26), April 12–18, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3744916.3787805

ICSE '26, Rio de Janeiro, Brazil

ACM ISBN 979-8-4007-2025-3/26/04
https://doi.org/10.1145/3744916.3787805

1 Introduction

Regression testing is a technique used to ensure that future code changes do not break the existing functionality of the current software version. It involves creating and preserving test cases that can be re-executed to validate the system's behavior as it evolves.

Creating a regression test case can begin with the current version of the code that is known to function correctly, using its existing behavior as a baseline for future validation. By capturing how the code behaves now, we can detect unintended changes introduced by future modifications. To strengthen the effectiveness of regression testing, it is important to *add test cases that cover a broader range of scenarios and execution paths*. This expanded coverage increases the likelihood of uncovering hidden bugs and ensures more comprehensive protection against regressions.

Recently, several researchers have explored the use of machine learning (ML) and large language models (LLMs) for automated test generation [3] and fuzzing in the context of regression testing [21, 42]. While having promising results, they largely depend on the reasoning capabilities of LLMs to generate test cases that exercise specific lines of code, conditions, or execution paths in the target program. Most existing techniques rely heavily on prompt-based querying of LLMs, which—according to a recent study by Chen *et al.* [6]—struggle with accurately reasoning about program execution. This limitation arises because LLMs are primarily trained on static code representations, lacking explicit runtime context or behavioral data. As a result, their ability to simulate execution, predict program states or code coverage is inherently limited.

In the context of coverage-guided test generation, one common consequence of this limitation is the phenomenon known as the *coverage plateau*: as test generation iterations continue, the rate of discovering new execution paths diminishes. For the LLM-based approaches, LLMs tend to produce test cases that fail to increase coverage, often generating different inputs yet covering the already-covered lines of code. Current approaches typically address this by simply *passive (re-)prompting* the LLMs *without offering helpful guidance*, which exacerbates the stagnation. This phenomenon has been observed and studied in prior works on coverage-guided test generation [3, 21], where coverage grows slowly over time.

In this paper, we propose TestWeaver, a coverage-guided, LLM-based approach for automated regression test generation that integrates lightweight program analysis (PA) to create a focused execution context that assists LLMs in better test generation. The goal is

to accelerate coverage growth by guiding LLMs toward generating higher-quality test cases that exercise previously uncovered lines of code. TestWeaver is built upon the integration of our core ideas:

First, instead of feeding the entire program—which may be long and complex—into the LLM, we extract and provide only the **backward slice** from the target line of interest. This slice includes the program statements that have a potential influence on the execution of that line. By narrowing the input to only the relevant parts of the code, we reduce the chance of hallucinations and minimize error propagation, while enabling the LLM to focus on the critical execution logic that affects the target line's reachability.

Second, instead of re-prompting the LLM to generate a new test case for all the not-yet-covered target lines (which may even require conflicting execution paths with one single test case), TestWeaver provides an **execution-aware, constructive feedback**. It leverages the *execution context from the current test suite*. The insight is that *the uncovered line might already be near the execution path of an existing test case*, even if not covered directly. Thus, giving the LLM the knowledge of execution behavior across the test suite offers a more informed basis to reason on what remains to be covered.

However, providing full execution information for all test cases is impractical due to LLM context window limitations. To address this, we draw inspiration from concolic testing [16, 36] and propose selecting **a "close" test case** to guide the LLM. Specifically, a test case T is close to a target line l_t if: 1) it executes a control-dependent condition of l_t; and 2) among such conditions, it covers the one that is closest in line distance to l_t. This approach exploits control dependence chains and execution traces to guide the retrieval structurally. This targeted strategy ensures that the LLM is grounded in relevant dynamic context, increasing the likelihood that the new test case will cover new code and avoid redundant explorations.

Finally, in each test generation iteration, we further equip the LLM with **execution in-line annotations**—a representation of dynamic program states. These annotations embed the values of relevant variables as in-line comments for each executed line at the key points during program's execution. This *execution-aware contextualization* allows the LLM to observe subtle data and control dependencies, *enabling it to better infer the values needed to steer execution along specific paths and ultimately reach the target line.*

We conducted an extensive empirical evaluation of TestWeaver. Using the CodaMosa benchmark—which includes 35 real-world Python projects and over 100,000 lines of code—we compared TestWeaver against two state-of-the-art baselines: CodaMosa [21] and CoverUp [3]. TestWeaver achieves **68%** line coverage and **62%** branch coverage, outperforming CoverUp (61% / 47%) and CodaMosa (46% / 23%). It generalizes well to highly-complex and longer programs, maintaining strong coverage and avoiding the coverage plateau common in the baselines by providing more constructive and execution-aware context to the LLM after each retry. Our tool also strikes a good balance between effectiveness and cost. It reaches the highest coverage point in 2.76× faster than CodaMosa while achieving higher coverage (68% vs 46%). In about the same time, it reaches higher coverage than CoverUp (68% vs 61%), with less token and money costs. Ablation study shows that removing any core component (backward slicing, closest-test retrieval, or execution in-lines) reduces coverage by up to 12%, confirming their critical role. In brief, we make the following contributions:

```python
def evaluate_sequence(arr: list[int]):
    score = 0
    freq = {}
    min_val, max_val = min(arr), max(arr)

    for x in arr:
        freq[x] = freq.get(x, 0) + 1
        score += x
    if len(arr) < 4:
        if arr[0] % 2 == 0:
            score += 4
        else:
            score -= 2
            if arr[-1] == 3:
                score += 10
    elif score % 5 != 0:
        if max_val - min_val > 50:
            score += 5
            if sum(1 for x in arr if x % 2 == 0) > len(arr) // 2:
                score *= 2
        elif 0 in freq:
            score += 7
    else:
        if arr == sorted(arr):
            score += 3
        elif all(v < 3 for v in freq.values()):
            score -= 1 # target line
        else:
            score += 6

    return score
```

Figure 1: LLMs struggle with execution reasoning: an example of GPT-4o's generating a test case to cover line 27.

(1) **TestWeaver: A Coverage-Guided Test Generation Framework** that integrates three ideas: (1) slicing shortens context by removing irrelevant code; (2) closest test cases enable contrastive reasoning, increasing the likelihood of covering new code while avoiding redundant exploration; and (3) execution paths and values provide richer feedback than simple coverage.
(2) **Focused Execution-Aware Context**: This feedback mechanism retrieves the most relevant test case and uses execution inlining to help the LLM better reason on program behaviors, and effectively generate test cases covering the target code.
(3) **Extensive Empirical Evaluation** We evaluate our tool on 35 Python projects against state-of-the-art baselines, showing superior code coverage, robust performance on complex code with less coverage plateau, and efficiency in runtime and token cost.

2 Motivation

2.1 Example and Observations

2.1.1 LLMs Struggle with Execution Reasoning. Let us use an example in Figure 1 to illustrate the issues and motivate our approach. In automated test generation for regression testing, the goal is to augment the existing test suite with new test cases that exercise a broader range of code lines and execution paths. Approaches leveraging large language models (LLMs) for this task must effectively guide the model to reason about program execution behavior. However, program execution prediction—also referred to as dynamic code reasoning—remains a significant challenge for LLMs [6, 20].

To illustrate this point, we examine a Python function that computes a score based on the input array `arr`. We prompted GPT-4o to

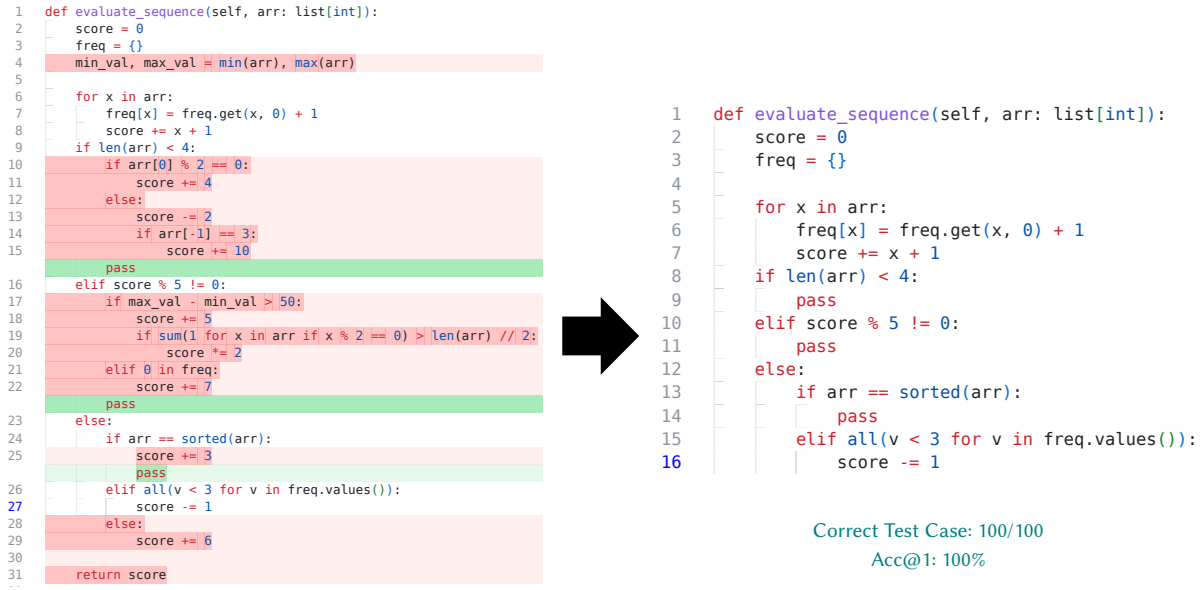

Figure 2: Improvement in test case generation using dynamic backward slicing

generate a test case specifically targeting line 27 and explain the code coverage, i.e., which lines have been covered for that generated test case. Execution of this line depends on several dynamic conditions, including the length of the array, whether the total sum is even, and whether the score exceeds a threshold of 10. Satisfying these conditions requires navigating through nested loops and conditionals—an inherently complex reasoning task for automated test generation. Despite the seemingly straightforward prompt, GPT-4o failed to produce any test case that would trigger line 27.

Observation 1 (LLMs Struggle with Execution Reasoning). *While LLMs generally perform well in several code-related tasks, they often struggle to reason about* actual code execution *and the dynamic evolution of the program state.*

In the example, the value of `score` evolves across multiple loop iterations and branches. Correctly generating a test case that reaches line 27 (`score -= 1`) requires reasoning about how this variable changes throughout execution. Since LLMs primarily rely on static syntax rather than execution semantics, they often overlook the interplay between conditions and runtime state.

To illustrate this, we prompted GPT-4o to generate a test case that would cover line 27. It produced the input `arr = [2, 3, 4, 6, 7, 9]` and explained that execution would skip line 16 (due to a miscalculation that `score = 19` satisfies `score % 5 == 0`) and line 24 (claiming the array is not sorted), which would supposedly lead to line 27 executing. However, in reality, the program did enter line 16, triggering line 19, which doubled `score` and changed the execution path, skipping line 27 entirely. GPT-4o's explanation reveals an incorrect mental model of control flow and variable states, highlighting its difficulty in reliably predicting dynamic execution, even in simple cases.

Li *et al.* [22] have reported that one contributing factor to the suboptimal performance of LLMs in code execution reasoning is their tendency to hallucinate when processing long or complex source code. To address this, we applied backward program slicing starting from line 27 and extracted the relevant statements that influence the program state at line 27. We then provided this slice to GPT-4o (Figure 2). As a result, GPT-4o successfully generated a test case which exercises that line. Moreover, as shown in Figure 2, when running multiple times, GPT-4o was able to produce 100 out of 100 test cases that cover line 27 using the sliced code, compared to only 9 out of 100 on the original complete program.

Observation 2 (Slicing Helps LLMs Generate Better Test Cases). *Program slicing helps focus the LLM's attention on the critical statements that influence a specific target line. This reduces distractions from unrelated code—such as loop constructs—and improves the LLM's precision in generating test cases covering a target line.*

The backward slicing isolates the `if` conditions around line 27, guiding the LLM to generate test cases more likely to cover line 27.

2.1.2 Coverage Plateau. During the iterative process of generating new test cases to improve code coverage, LLMs typically identify many new execution paths early on, leading to a rapid increase in code coverage. However, as the process continues, the rate of discovering new paths declines. The remaining unexplored paths tend to be more complex, often requiring specific and rare input conditions to be triggered. As a result, coverage growth slows significantly, leading to a phenomenon known as the coverage plateau [3, 21].

The limited reasoning ability of LLMs for dynamic program behavior is one cause of the coverage plateau problem. Another key factor is that existing LLM-based test generation approaches [3] do not provide constructive feedback to help the LLMs. Instead, they repeatedly prompt the LLM to cover all remaining uncovered lines L *without offering any guidance to help the model improve in the*

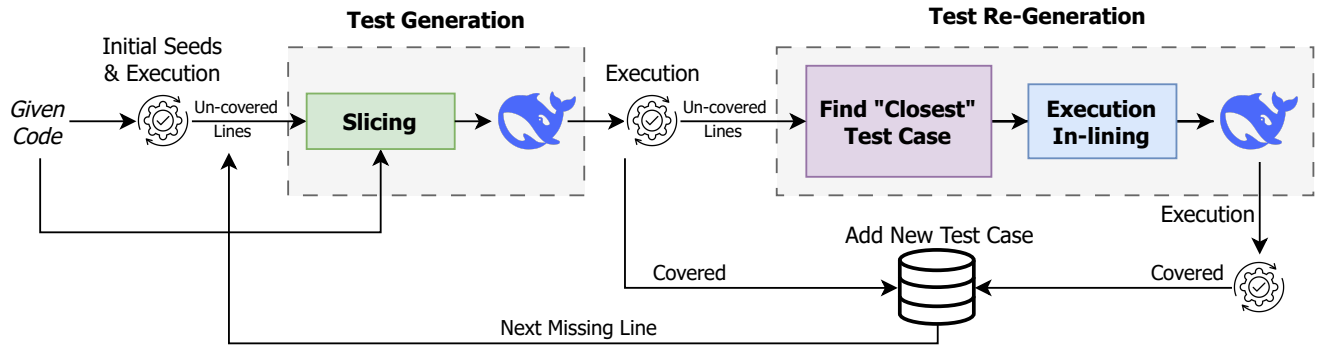

Figure 3: Overview of the TestWeaver pipeline. The process begins by generating initial test seeds and identifying uncovered lines through execution. For each uncovered line, a backward slice is computed to produce a sliced code, which is then provided to the LLM for test generation. If the generated test fails to cover the target, the re-generation stage is triggered: the closest failing test is retrieved from the current suite, its execution trace is used to annotate the sliced code with variable values (execution in-lines), and the enriched prompt is re-submitted to the LLM. Any successful test is added to the suite, and the process continues until all uncovered lines have been addressed or a time limit is reached.

next attempt—a strategy we call **passive feedback**. This passive approach has critical shortcomings. Providing the entire set of uncovered lines L can easily confuse the LLM, since covering multiple lines may require conflicting execution paths within a single test case. Simply repeating the same coverage request without any context often fails to help the LLM make progress, directly contributing to stagnation in coverage growth. This passive feedback does not help the LLMs recognize the nuances in the execution with different test cases, in which LLMs by themselves have limited capability.

Observation 3 (Coverage Plateau). *The code coverage plateau problem is caused by the limited capability of LLMs in dynamic code reasoning and the passive feedback strategy in the state-of-the-art, LLM-based regression test case generation approaches.*

We hypothesize that *providing execution data—such as execution traces or symbolic execution results*—can significantly improve an LLM's understanding of dynamic program behavior, enabling more effective and targeted test case generation. For example, dynamic symbolic execution can produce path constraints that describe how to reach a target statement, as in concolic testing [16, 36]. Yet, solving these constraints, especially when they involve multiple data types, is inefficient or even infeasible for SMT solvers. On the other hand, supplying execution information for the entire test suite and its covered lines will exceed the LLM's context window limits.

Observation 4 (Execution-aware Feedback). *Providing an LLM with execution information on the current test suite could help it generate test cases for uncovered lines.*

2.2 Key Ideas

From the observations, TestWeaver is designed with the following:

*2.2.1 Key Idea 1. [***Program Slicing** *for Targeted Test Case Generation].* Program slicing reduces code complexity by isolating the relevant portions that influence a specific target line. By extracting only the essential information needed for test case generation, slicing helps the LLM concentrate on the most critical parts of the code. This focused context minimizes the risk of hallucination and ensures that generated test cases align more closely with the actual execution flow. Additionally, slicing narrows the search space, making the LLMs to generate test cases more targeted and efficient.

*2.2.2 Key Idea 2. [***"Closest" Test Selection** *to the Target Line].* We aim to leverage the coverage information from the entire test suite to guide test case generation. However, encoding all test cases along with their covered lines and associated program states exceed the context window limitations of LLMs. To address this, we propose a strategy that uses the full test suite to identify a subset of similar test cases—those that cover execution paths "closest" to the target line (Section 3.3). By focusing on this similarity-based subset, we can guide the LLM to generate new test cases that meaningfully extend coverage without duplicating prior explorations. This approach mitigates the risk of out-of-context generation or hallucination and allows for more precise and efficient targeting of uncovered lines.

2.2.3 Key Idea 3. [Enhancing Code Execution Understanding with **Inlining Execution Data***].* To enhance an LLM's ability to reason about code execution and generate test cases that cover specific lines of code, we integrate the LLM with external tools to support in-line execution feedback. Specifically, after executing a generated test case, we annotate the corresponding lines with the observed program states including the runtime values of the variables involving in the statements as in NeXT [30]. This enables the LLM to track the variable states and data flows in "execution". Inspired by agent-based systems, this approach allows the LLM to simulate actual program execution more effectively, resulting in more accurate and reliable generation of test cases to cover the target line.

3 TestWeaver Test Generation Workflow

3.1 Overview

Our pipeline (Figure 3) for automated test generation begins by producing an initial seed test suite, which can be generated by any initial corpus test generation algorithm [17, 31]. In our experiment, we leverage LLM to generate N initial test cases. Then, all the lines

that remain uncovered are identified. These uncovered lines then drive the slice-guided test generation step: for each target line, a backward slice (Section 3.2) is computed to isolate the minimal fragment of source code that influences its execution. That fragment—together with the target line itself—is presented to a LLM, whose attention is thus focused solely on the relevant control and data dependencies when generating a new test case. Specifically, when the generated candidates still fail to cover the target line after a predefined number of attempts, our tool invokes the execution-inlining test regeneration step. A heuristic retrieval method selects from the existing suite the "closest" test case (Section 3.3) whose execution trace passes through the nearest conditional statement of the target line but thereafter diverges along a different path. The backward slice of that "closest" test case is then instrumented with the inline annotations of runtime values observed under that test case, providing the LLM with concrete execution contexts. These enriched slices are iteratively presented to the LLM until the target line is covered or a predefined number of iterations is reached. Any successful test is added to the test suite, and its coverage is subsequently used to address the remaining uncovered lines.

3.2 Test Generation with Program Slicing

After identifying a target line of code to be covered, TestWeaver computes a dynamic backward slice to keep only the minimal number of statements that influence the execution of the target line. Because there does not exist any dynamic program slicing library for Python code that fits our workflow, we *re-implemented* Agrawal and Horgan's dynamic backward slicing algorithm (Approach 2) [1]. Additionally, because that algorithm supports only intra-procedural slicing, we also *extended it to perform inter-procedural backward slicing* through multiple involved functions. Let us detail it.

In the first phase, TestWeaver builds the static program dependence graphs (PDGs) for all functions and module-level statements encountered. Next, to extend Agrawal and Horgan's algorithm to support inter-procedural flows, it constructs a system dependence graph (SDG) by linking these PDGs of those functions via the call sites (and actual arguments), the function entries (and formal arguments), `return` statements, and variable assignments and their sources. Initially, all the SDG nodes (statements) and edges (program dependencies) are unmarked (i.e., not-executed). In the next step of this phase, following the execution of the given test case, our algorithm marks those SDG nodes and edges whose dependencies arise as executed including the inter-procedural dependencies.

In the second phase, i.e., the slicing phase, from the target statement, the algorithm traverses the SDG backward only along the marked nodes and edges (i.e., the executed statements and actual control and data dependencies, including match-case patterns). This helps TestWeaver include only *the statements that were executed and came from the executed path.* Specifically, in addition to the data and control dependencies, the algorithm follows different types of relations among variables and references [22]: (1) `def-use` (definition to reference): a definition and a reference to a variable have a `def-use` relation if there exists a control flow from the statement containing that definition to the statement containing that reference without any intervening redefinition of that variable; and (2) `info-flow` (reference to definition): for a given statement S, a reference to a variable v_1 has an info-flow relation with the definition of another variable v_2 if the value of the variable v_1 on the entry to the execution of the statement S may affect the value of v_2 on the exit from the statement S. The nodes (i.e., statements) that have been reached via these relations are collected into the resulting dynamic backward slice from the target statement.

Note that, during this traversal of the slicing phase, the algorithm also propagates/expands the slice through the actual function calls and returns in order to perform inter-procedural slicing.

This program slice and the target line are presented to the LLM with the test generation prompt in Table 1.1. This guides the LLM to focus on relevant statements and their dependencies to the target line to generate a test case covering it. If the generated test case from the LLM is un-executable due to compile errors, we use the same program analysis in CoverUp to guide the LLM in fixing it. After a limited number of attempts of fixing, we drop that test case.

3.3 Retrieving "Closest" Test Case to Target Line

3.3.1 Formulation. Given a target line l_t in the program that remains uncovered, we aim to identify a test case T from the current test suite $\mathcal{T}$ that has traversed the code *as "closest" as possible* to l_t. Instead of relying on full execution path comparison, we leverage the *control dependencies* of the line l_t, specifically, the conditional constructs (e.g., `if`, `while`) needed to be satisfied for l_t to execute. Let:

- $\text{Cond}(l_t)$ = the ordered list of control-dependent conditional statements governing l_t, sorted by proximity (in source line number) to l_t
- $\text{Exec}(T)$ = the set of lines executed by test case T

Then, a test case T is considered *close to* l_t if it satisfies:

(1) **Conditional Coverage Proximity**: The test case T is considered close to l_t if it executes a condition statement $c \in \text{Cond}(l_t)$ such that the line distance $|c - l_t|$ is minimized:

$$\text{Closeness}(T, l_t) = \min_{c \in \text{Cond}(l_t) \cap \text{Exec}(T)} |c - l_t|$$

(2) **Structural Control Dependency**: Only the condition statements that appear in the control flow of l_t are considered. This ensures that the test case follows an execution path that nearly reaches the target line l_t-diverging only at its closest condition statement—provides valuable execution context for the LLM to generate a test case covering l_t.

In summary, a test case T is **close** to a target line l_t if it executes a control-dependent condition of l_t, and among such conditions, it covers the condition that is closest in line distance to l_t. This approach exploits control dependence chains and execution traces to guide the retrieval structurally.

3.3.2 Algorithm. Algorithm 1 shows our retrieval strategy based on conditional coverage proximity. It begins by identifying the set of control-dependent conditions for the target line l_t using static analysis over the control dependencies (line 1).

We initialize the closest test case T^* as `None`, and set the best (minimal) line distance $\delta_{\min}$ to ∞ (line 2). We iterate over each test case T in the current suite $\mathcal{T}$ (line 3). For each test case, its dynamic execution trace E_T is obtained (line 4), which is the set of lines covered during the execution. We then loop through each condition c that controls l_t (line 5). If a condition c is executed by T (line 6), the algorithm computes the absolute line difference $\delta = |l_c - l_t|$ (line 7).

Algorithm 1 FindClosestTestCase($\mathcal{T}$, l_t, CDG)

Require: Test suite $\mathcal{T}$, target line l_t, control dependency graph CDG
Ensure: Closest test case $T^* \in \mathcal{T}$ to target line l_t

```
1:  Cond(l_t) ← GetControlConditions(l_t, CDG)
2:  T* ← None,   δ_min ← ∞
3:  for all test case T in T do
4:      E_T ← GetExecutedLines(T)
5:      for all c ∈ Cond(l_t) do
6:          if c ∈ E_T then
7:              δ ← |l_c − l_t|
8:              if δ < δ_min then
9:                  T* ← T
10:                 δ_min ← δ
11:             end if
12:             break ▷ Only consider closest conditional covered
13:         end if
14:     end for
15: end for
16: return T*
```

Helper Functions:
GetControlConditions(l_t, CDG): Returns the ordered list of conditions controlling l_t, based on the CDG.
GetExecutedLines(T): Returns the set of all program lines executed by test case T.

If this δ is smaller than the current best distance $\delta_{\min}$ (line 8), we update T^* as the new closest test case and record δ as the new minimum (lines 9–10). Since the conditions are ordered by proximity to l_t, we break early after the first match is found (line 12).

The `GetControlCondition()` function (Algorithm 1) and the dynamic backward slicing algorithm support the following control-flow constructs in Python: `if`, `while`, `for`, `try-catch`, `with`, `try-except`, `finally`, `match-case`, `break`, `continue`, and `return`.

3.4 Test Re-generation with Execution In-lines

Only after extracting the program slice on the closest test case relevant to a given uncovered line, we augment it with execution information in the form of *execution in-lines*. Note that the initial program slice from the Test Generation module (Figure 3) does not contain execution in-lines. They are the annotations that explicitly encode the runtime values of variables at each statement, thereby exposing the concrete program state to the LLM. The annotations also encode the execution index of a statement when it was visited (i.e., the k-th statement executed). The goal is to help the LLM understand how the execution flows under a test case. We expect that to help the LLMs to derive how it might need to change the input to cover the target line.

To obtain these values, the closest test case is first identified from the current test suite using Algorithm 1. We execute the closest test case and the dynamic backward slice from the target line is obtained. Then, we use a tool to record the execution information and instrument the backward slice with *execution in-lines*. That is, the inline comments are inserted after each executable line to show the values of in-scope variables at the corresponding statement. The format of each in-line annotation is as follows:

```
# (k) var1 = v1; var2 = v2; ...
```

Here, (k) denotes the execution index, and each `var=value` pair reflects the variable's value after executing that line. For example,

```
x = a + b    # (1) a = 2; b = 3; x = 5
if x > 10:   # (2) x = 5
```

These in-lines follow the style proposed by the NExT framework [30], which showed that providing such structured and localized execution context can significantly improve the reasoning capabilities of LLMs in program execution understanding tasks.

The structure of the prompts are outlined in Table 1. By supplying both the logical structure of the code and the actual values encountered during execution, the LLM is provided with a dual perspective: the static control and data flow from the backward slice, and the dynamic information under a nearly-covering test case. This enriched input guides the LLM to generate a new test case that corrects the previous failure, often by flipping the one condition needed to

Table 1: Prompt templates for TestWeaver

1. Prompt Template for Test Generation phase

Please generate a single pytest test function for the class '{class_name}' based on the provided code under test. Your response should include only one test input.
Program under test:
—-
{code_slice}
—-
The code above does not achieve line: {target_line}
<instructions>
1. Create a new pytest test function…
…
11. IMPORTANT: Your test method should begin with…
Note: If the class under test is an abstract class, do not instantiate it directly in your test. Instead, create a concrete subclass that implements all abstract methods before instantiation, or only test static/class methods without instantiating the abstract class.
</instructions>

2. Prompt Template for Test Re-Generation phase

Given the following Python program and the function '{func_name}' within the class '{class_name}', your task is to write a test case that executes the specific line of code: {target_line}.
You are provided with a closely related test case that nearly covers line {target_line}:
{closest_test}
Additionally, here is a cheatsheet containing the gold execution trace for the above example test case. You may use this information to inform your reasoning, but present your analysis as if you derived it independently.
{code_slice_with_exec_inlines}
<instructions>
Using the provided closest test case, please:
1. Analyze why the given test case does not execute the target line.
2. Outline a step-by-step strategy to ensure that line is executed.
3. Finally, write a new test case that successfully triggers the execution of the target line.
Your generated test case should begin with the following template…
Please structure your response in two distinct sections:
- Enclose step-by-step reasoning within '<thinking>' tags.
- Enclose final test case within '<answer>' tags, using backticks for formatting.
Important:
- If the class under test is abstract, do not instantiate it directly. Instead, create a concrete subclass that implements all abstract methods before instantiation, or only test static/class methods without instantiating the abstract class.
</instructions>

reach the target line that needs to be covered. This idea is inspired by the Concolic Testing approach [16, 36], where conditions in a path constraint are flipped to generate new test cases that explore alternative execution paths. However, instead of relying on a SMT solver to find inputs that satisfy the modified constraint, we use the LLM to generate the new test case directly.

After a target statement is covered or skipped, multiple uncovered statements may remain. TestWeaver sorts those not-yet-covered statements according to their appearance order in the execution trace and select the first one in that order.

4 Empirical Evaluation

For evaluation, we seek to answer the following questions:

RQ1. [Effectiveness in Test Generation] How effective in code coverage is TestWeaver in generating test cases in comparison with the baselines CoverUp [3] and CodaMosa [21]?

RQ2. [Efficiency in Test Generation] How efficiency is TestWeaver in increasing code coverage during test generation compared to the existing approaches?

RQ3. [**Ablation Study**] How critical do the components of our method contribute to its performance?

RQ4. [**Cost Efficiency**] How efficient is TestWeaver in coverage-guided test generation in terms of token costs?

We use `deepseek-v3-0324` [23] for lower costs, which serves as the base LLM for all variants including baselines.

Datasets. We conduct our evaluation on the CodaMosa (CM) suite [21], a dataset derived from 35 open-source Python projects previously used in the evaluations of BugsInPy and Pynguin. It has about 100,000 lines of code across 425 Python modules. We did not use the Pynguin (PY) [26] and MuTAP (MT) [10] datasets because the CM suite is 20× and 50× larger than PY (5,000 LOCs, 84 modules) and MT (2,000 LOCs, 163 functions), respectively.

Baselines. We compare TestWeaver with the two baselines: coverup [3] and CodaMosa [21]. We reused their core pipelines and replaced GPT-4o services with DeepSeek for cost-efficiency.

Procedure. In the initial test seed phase, we used DeepSeek to generate 10 test cases. In practice, they can be generated by any initial corpus test generation algorithm [17, 31]. During the subsequent test generation phase, which incorporates backward slicing, up to 6 retries are permitted per target line, with each retry representing a single LLM API call aimed at covering that line. In the re-generation phase, any lines that remain uncovered are revisited, allowing up to 5 retries per line. All generated test cases are executed in isolation to prevent state interference and to ensure precise measurement of coverage. For all approaches, we ran for a maximum of 6 hours.

Metrics. We used three standard metrics for evaluation: *line coverage*, *branch coverage*, and combined *line+branch* coverage. line+branch coverage = (#covered_lines + #covered_branches) / (#lines + #branches). Aggregated metrics for each type of coverage are reported as the overall coverage across the entire test suite.

5 Effectiveness of Test Case Generation (RQ1)

Figure 4 presents the aggregate coverage for the entire benchmark suite, where each value denotes the ratio of the covered lines/branches to the total ones *across* the files of 35 projects in our dataset. This applies consistently to line, branch, and line+branch.

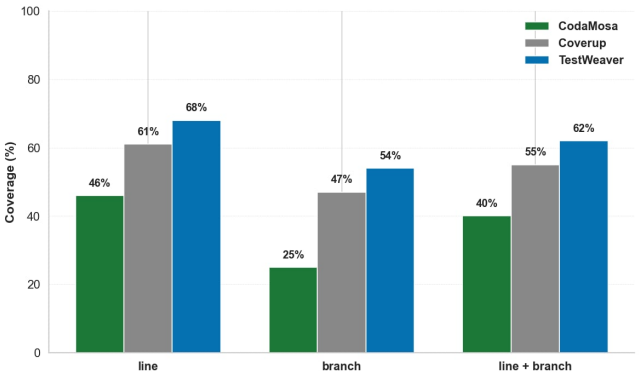

Figure 4: Performance comparison on code coverage (RQ1)

Table 2: Stratified coverage result by repository size (RQ1)

Module	Metric	CodaMosa	CoverUp	TestWeaver
Low-#lines	Line	50%	69%	**73%**
	Branch	38%	66%	**69%**
	Line + Branch	45%	68%	**72%**
Mid-#lines	Line	44%	62%	**67%**
	Branch	27%	53%	**57%**
	Line + Branch	39%	59%	**64%**
High-#lines	Line	41%	52%	**65%**
	Branch	23%	43%	**51%**
	Line + Branch	36%	47%	**57%**

Across all settings, as seen in Figure 4, TestWeaver consistently achieves higher coverage than both baseline methods. On the entire dataset, it achieves 68% line coverage, outperforming CoverUp and CodaMosa, which achieve 61% and 46%, respectively. Similarly, for branch coverage, TestWeaver reaches 54%, compared to 47% and 25% for the respective baselines. In terms of line+branch coverage, TestWeaver achieves 62%, whereas coverUp and CodaMosa reach 55% and 40%, respectively. In brief, within the same number of hours per repository, TestWeaver achieves more line and branch coverages than the two baselines CoverUp and CodaMosa.

5.1 Stratification based on Repository Size

We aim to study how TestWeaver covers the source code in the repositories with different sizes in comparison with the baselines. To this end, we partition the dataset into three groups according to the number of lines of code in each file: Low-#lines (0–150 lines), Mid-#lines (150–500), and High-#lines (500–1, 100). The total LOCs in each group are non-trivial: Low (18,126), Mid (27,187), and High (10,209). This allows us to examine how models' performance scales with project size, which is crucial for understanding generalizability.

As shown in Table 2, TestWeaver consistently achieves the highest coverage results among all the approaches across three categories. In low-complexity modules, it outperforms the baselines by margins of 4%-31% across all metrics. This performance advantage persists in both medium and high number-of-line groups, where it maintains a higher coverage from 4%-13% over CoverUp and 14%-30% over CodaMosa.

All approaches have experienced a moderate decline in code coverage for the entire dataset as the source code has more lines

of code, which causes LLMs to navigate through more predictive execution. However, TestWeaver remains notably more stable. Its performance degrades only slightly, indicating that the improvements generalize well for large repository sizes.

This robustness arises from our core design choices. First, our backward slicing strategy significantly reduces the size of input to the LLM by retaining only statements relevant to the uncovered targets. To further assess its effectiveness, we collected the *lines that CoverUp consistently fail to cover—referred to as difficult lines. TestWeaver is able to cover nearly 20% of these difficult targeted lines*, with the slicing ratios from 13%-21%. This demonstrates that backward slicing enhances LLM to pay attention to the most influential statements, mitigating the risk of hallucinations caused by irrelevant or noisy statements and computations—an issue that becomes more severe in large and complex codebases. Second, our constructive feedback mechanism, which selects semantically closest test cases as in-context exemplars, guides the LLM with a more focused context in generating more targeted and logically consistent test cases. By observing concrete parameter values and execution paths, the LLM can better infer which conditions must be altered to reach uncovered lines – all without actual execution.

Together, these mechanisms enable TestWeaver to scale more effectively with project size, while enhancing precision and reducing erroneous generations—a key advantage over the baselines.

5.2 Stratification based on Code Complexity

Sometimes, the number of lines does not reflect well the complexity of the actual execution. Therefore, we further evaluate TestWeaver's effectiveness by stratifying all the functions based on their *cyclomatic complexity (CC)*. CC quantifies the structural complexity of a program's control flow. A CC value above 50 is generally considered *hard and complex*, as it indicates dense branching and deeply nested logic that are challenging for both human reasoning and automated test generation. To assess whether TestWeaver remains more effective than the baselines under different complexity levels, we divide the function into four groups with different ranges of CC values: [1–50], [50–100], [100–200], and [200–300]. This stratification allows us to observe how the performance of the approaches scales with increasing code complexity.

Table 3: Code Coverage across Functions Stratified by Cyclomatic Complexity (CC) (RQ1).

CC Range	CodaMosa	CoverUp	TestWeaver
1–50	50%	68%	**73%**
50–100	44%	65%	**68%**
100–200	39%	55%	**63%**
200–300	45%	57%	**66%**

As seen in Table 3, *TestWeaver consistently outperforms both CoverUp and CodaMosa across all complexity groups*. The performance gap becomes more pronounced as complexity increases, highlighting TestWeaver 's robustness in handling intricate control flows. This improvement stems from its program slicing and execution-inlining strategies, which help the LLM focus on relevant control and data dependencies—reducing distractions and hallucinations even in complex scenarios. In our ablation study (Section 7), we will show that with our execution-inlining and closest test retrieval, TestWeaver handles better with intricate code.

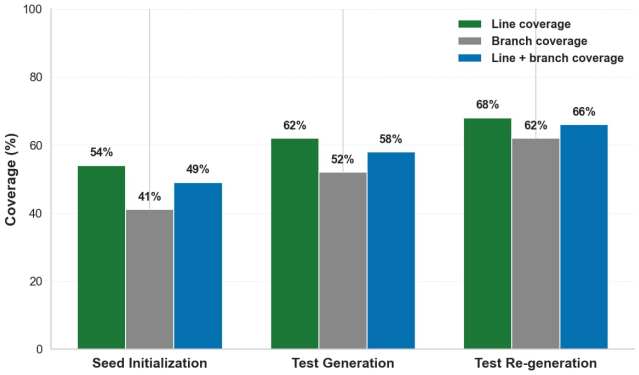

Figure 5: Coverage improvements across three phases of TestWeaver on the CM suite (RQ2).

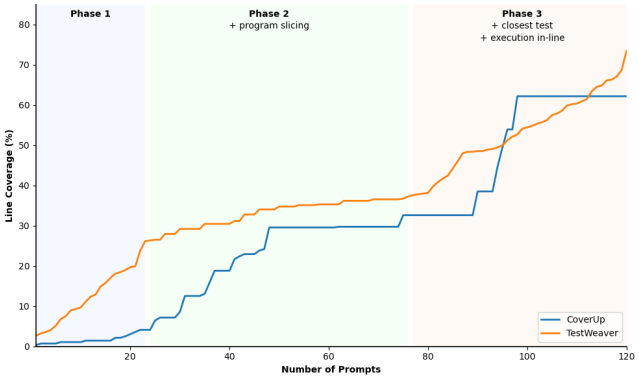

Figure 6: Line coverage progression across 120 prompts for `typesystem.fields` module. Colored areas show phases (RQ2)

6 Efficiency of Test Case Generation (RQ2)

In this experiment, we aim to investigate how code coverage evolves throughout three phases of test generation in TestWeaver: (1) *Seed Initialization*, (2) *Test Generation*, and (3) *Test Re-Generation*.

As seen in Figure 5, the coverage starts at 54% after the initial phase and increases to 62% during the second phase, highlighting the effectiveness of our backward slicing strategy, trimming down the irrelevant statements to help the LLMs focus on the important ones. By limiting the LLM's attention to only the code statements relevant to the uncovered targets, slicing reduces the input size to just 69–78% of the original code, resulting in more focused and effective prompt construction. This result confirms our Key Idea 1 in our design listed in Section 2.2.1. In the third phase, coverage rises further to 68%. While slicing narrows the context, LLMs may still struggle with complex control flows or variable interactions. To address this, TestWeaver incorporates our feedback mechanism, specifically provides additional guidance including the "closest" test cases and inlined execution information. These two feedbacks help the LLM better infer execution semantics and reduce hallucination.

Table 4: Comparison of retry strategies in CoverUp and constructive feedback mechanisms in TestWeaver (RQ2).

Aspect	CoverUp	TestWeaver
Constructive feedback	Listing all non-covered lines in prompt	Providing "closest" test case and inline explanation
Execution awareness	Ignores runtime execution data	Grounded in actual execution via inline traces
Coverage progression	More coverage plateaus due to weak feedback	Consistently improves with targeted guidance
Code reduction strategy	Function-level segmentation only	Line-level backward slicing to exclude irrelevant code

Comparative Analysis. Notably, our tool remains more effective even on highly complex modules in the CM suite. Let us use a case study on the module `typesystem.fields`, exceeding 550 lines with a cyclomatic complexity over 350. We further illustrate this improvement by comparing *per-prompt coverage progression* between TestWeaver and the best baseline, CoverUp (Figure 6). The X-axis shows the number of prompts, while the Y-axis shows the line coverage in percentage.

To compare with the best baseline CoverUp (Figure 6), TestWeaver consistently achieves higher line coverage with each prompt, while CoverUp frequently encounters coverage plateaus. Notably, its longest plateau persists for about 30 consecutive LLM retries. After reaching its maximum coverage at the 98th prompt, CoverUp stalls at 63% coverage, even after more than 20 additional retries. In contrast, TestWeaver continues to make progress: it surpasses 63% at the 112th prompt and reaches 75% by the 120th prompt.

A closer look at the results shows that this improvement comes from (1) the reduced number of statements the LLM must handle thanks to backward slicing in both the initial and test generation phases, and (2) the *constructive feedback strategies* applied during retries in the final test re-generation phase.

CoverUp uses a *more passive retry strategy, simply listing all uncovered lines in a prompt to the LLM for another attempt.* This provides no guidance on how to cover those lines, which may even require conflicting execution paths within a single test case. As a result, the LLM lacks critical execution context and is forced to guess, often repeating the same mistakes. In Figure 6, *in the total of 120 prompts to get to 63% of line coverage, 69 re-prompts to the LLM from CoverUp did not result any new line covered. The top-3 coverage plateaus in that example lasted 14, 20, and 30 prompts.*

In contrast, TestWeaver offers more constructive feedback by retrieving the "closest" test case and embedding execution traces directly into the prompt. This richer, execution-aware feedback helps TestWeaver recover from failures and make continuous progress. Additionally, unlike CoverUp, which segments code only at the function level, TestWeaver performs fine-grained program slicing to exclude lines that do not affect the target line's execution. This reduces unnecessary context and minimizes hallucination. Consequently, while CoverUp often hits an early coverage ceiling due to its shallow feedback loop and coarse context selection, TestWeaver reduces this plateau by providing precisely the information needed to resolve each uncovered line. In Figure 6, TestWeaver has only 15 times that retries did not result in any increasing coverage. Table 4 summarizes the key improvements of TestWeaver over the baseline, with the maximum coverage plateaus of 5 prompts.

Table 5: Ablation on test generation: overall coverage obtained on the CM Suite (more is better) (RQ3)

% Coverage	Line	Branch	Line + Branch
TestWeaver	**62%**	**52%**	**58%**
– w/o Slicing	58% (-4%)	45% (-7%)	52% (-6%)

Table 6: Ablation for test re-generation: overall coverage obtained on the CM Suite (more is better) (RQ3)

% Coverage	Line	Branch	Line + Branch
TestWeaver	**68%**	**62%**	**66%**
– w/o Execution Inlining	63% (-5%)	56% (-6%)	61% (-5%)
– w/o Closest Test Sel.	62% (-6%)	52% (-10%)	58% (-8%)

7 Ablation Study (RQ3)

TestWeaver has three key components: (1) backward slicing in the test generation phase, (2) "closest" test case selection, and (3) execution inlining in the test re-generation phase. In this experiment, we study their contributions by disabling each of them.

Backward Slicing. Table 5 presents the effect of disabling backward slicing. While program slicing improves coverage by 4%–7% overall, this gain reflects its ability to assist the LLM in reducing hallucination. We further analyze *the cases where test generation fails to cover the target line without slicing but succeeds with slicing; in these, slicing eliminates 16%–25% of the original code per file, allowing the LLM to focus on fewer, more targeted paths.* This result corroborates our Key Idea 1 on using program slicing.

Closest Test Selection and Execution Inlining. Table 6 illustrates the impact of the closest test selection and in-line execution components. The results show that removing either component consistently degrades coverage. Among them, removing the closest test mechanism leads to the most noticeable drop—between 6%—10% across coverage metrics—while excluding in-line execution results in a 5%–6% decrease. Notably, the two factors are complementary: leveraging a closest test case and adding execution information provide the LLM with dynamic signals, leading to TestWeaver's most effectiveness in generating test cases to cover target lines. This result corroborates our Key Ideas 2–3 in leveraging the closest test case and execution data to generate better test cases.

Test Selection Strategies. Table 7 presents an ablation study evaluating the importance of the closest test case selection strategy in the test re-generation phase. In the standard setting, TestWeaver identifies the "closest" test case to the target line (yet not covering it), and uses its execution trace to annotate the program slice with execution in-lines. This configuration achieved 68% line coverage for entire dataset. In contrast, replacing the closest test case with a *randomly selected test case* from the current test suite reduced coverage to 65%, indicating that *an arbitrary execution context provides weaker guidance to the LLM, leading to lower code coverage.*

A second variant incorporated execution traces from k random test cases (k=5) into the prompt, without prioritizing proximity to the target line. This multi-trace setup achieved 64% coverage—lower than both the random baseline and the closest-test case strategy. This suggests that execution in-lines derived from the "closest" test

Table 7: Ablation for "closest" test selection strategies (RQ3).

% Coverage	Line
Closest Test Case	**68%**
Random Test Case	(-3%) 65%
k Random Test Cases	(-4%) 64%

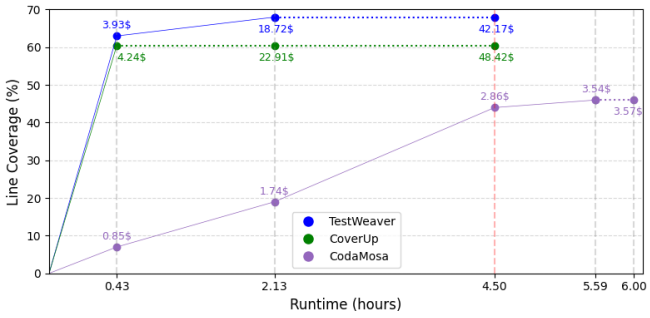

Figure 7: Coverage progression over running time with token cost in USD. A dotted line indicates coverage plateau after last saturation, where no further improvement occurs (RQ4).

case play a critical role in guiding the LLM in effective test generation to cover the target line. This also confirms the effectiveness of our feedback with test cases and "closest" test selection algorithm.

8 Efficiency on Token Costs (RQ4)

In this experiment, we aim to assess TestWeaver's time and cost efficiency in comparison with those of the two baselines. Since CoverUp's authors set the running time of 4 hours in their experiments, we executed TestWeaver and CoverUp for 4.5 hours for a fair comparison. Since CodaMosa's coverage increases more slowly, we let it run up to 6 hours to observe coverage changes.

To assess time and cost efficiency with regard to coverage, along the execution of a tool, we tracked *the last saturation point–defined as the time where the further (remaining) retries yield no additional line coverage.* The last saturation point corresponds to the last coverage plateau for a tool's run, allowing us to assess *a tool's peak coverage and the earliest time* at which it is reached. We recorded the following metrics for comparison: the current running time, the (token) cost in USD for LLM usages, and the achieved coverage. We recorded those metrics at the last saturation point and at the run end for a tool, and the respective values of the other tools.

Last Saturation Point and Coverage. As seen in Figure 7, at the ending time, TestWeaver achieves *the highest coverage for the entire dataset* (68%) (compared to CoverUp (61%) and CodaMosa (46%)).

Compared to CoverUp, which hits its last saturation early at 0.43 hour, TestWeaver has reached 62% coverage by that time—at lower cost. At its last saturation (2.13), TestWeaver reaches higher coverage (68%), and has a shorter last plateau (i.e., no further gains, blue vs. green dotted lines). In contrast, CodaMosa has a later last saturation point (5.59 hours) than TestWeaver, yet attains only 46% coverage, even with more time. *This shows TestWeaver's better coverage performance at all checkpoints (last saturation and ending).*

This plot *does not have the coverage plateaus from a tool before the last saturation point* since it aims to show only efficiency with regard to the ***total coverage*** of all repositories in our entire dataset. In our experiment, at any time before the last saturation point, there exists *one or more repositories in the dataset with increasing coverages*, leading to *increasing total coverage*. Thus, *this timeseries plot for the total coverage on the entire dataset does not contain the coverage plateaus before the last saturation.* Moreover, large repositories need more running time. Thus, for coverage plateaus, in Section 6, we showed them for a complex repository on the prompt basis. The coverage plateaus for other repositories [37] followed similar trends as in Figure 6. In brief, this plot with last saturations shows the time to the peak coverage at the last plateau, rather than all plateaus.

Cost and Coverage. Despite having lower token costs due to no constructive feedback as in TestWeaver, CodaMosa's coverage progresses more slowly, requiring the entire 6 hours to reach its final coverage of only 46%, compared to TestWeaver's 2.13 hours with 68%. At any checkpoint, CodaMosa's coverage is *much lower than* TestWeaver's. Thus, *the tradeoff—higher token cost than CodaMosa in exchange for superior coverage— is justified for TestWeaver.*

At 0.43 hour, TestWeaver already attains 62% coverage with $3.93 compared to CoverUp's only 61% with $4.24. By 2.13 hours, it reaches 68% coverage at a cost of $18.72, while CoverUp remains stuck at 61% even after running for the same duration at a higher cost ($22.91). Extending CoverUp's running to 4.5 hours results in no further gain, as coverage remains at 61% with a cost of $48.42. TestWeaver's lower cost compared to that of CoverUp stems from its shorter, sliced prompts, which reduce token usage while keeping the LLM focused on relevant execution paths. In brief, TestWeaver advances CodaMosa and CoverUp in terms of coverage and cost.

9 Limitations and Threats to Validity

Limitations: First, our re-implementation of backward slicing may omit essential contexts from dependent classes, potentially leading to incorrect type inference for method parameters. Second, similar to CoverUp, LLMs sometimes generate un-executable test code. We use the same program analysis module in CoverUp to resolve the compilation errors. Third, in cases where the target line is hard to reach, no closest test case may be found—leaving the LLM without a strong exemplar to guide generation, hindering effectiveness. Finally, after a fixed number of attempts with "closest" test case and execution inlines to cover a line, we move on to the next line.

External Validity: Our chosen benchmarks might not be representative, but they have been used in the existing work, enabling a fair comparison. We evaluated TestWeaver with `deepseek-v3-0324`. The results may vary for other LLMs. We evaluated TestWeaver only on Python. More experiments are needed for other languages.

Internal Validity: The third-party tools for dependence analysis or the compiler, may introduce errors. While the code in our dataset may be publicly available, very few associated test cases are accessible. Furthermore, the available inputs are limited in scope and domains. Due to the lack of comprehensive source code coverage and the infeasibility of exhaustively executing all possible inputs, *the risk of data leakage is minimal.* As such, the soundness of our evaluation remains unaffected. The network traffics, latency, server loads when interacting with LLMs affected the number of API calls to the LLMs. Finally, potential limitations in our re-implementation of a dynamic backward slicing algorithm can introduce bugs.

Construct Validity: Varied prompts may lead to varied input distributions, potentially affecting results. To address this, we define the prompts with well-specified structure as shown in Table 1.

10 Related Work

TestWeaver is closely related to CoverUp [3], which combines coverage analysis, code context, and iterative feedback in prompting LLMs to improve code coverage. However, TestWeaver introduces key advancements. First, TestWeaver provides more focused context to the LLM by leveraging backward slicing from the target line, whereas CoverUp supplies function-level code segments. Second, CoverUp only considers passive feedbacks with all the uncovered lines and retries the prompting without any helpful guidance. In contrast, TestWeaver augments the prompt with execution inlines that expose relevant variables and program state, thereby helping LLMs reach the target line. Finally, CoverUp uses PA to fix the syntactically incorrect generated test code. We leverage PA to improve both incorrectly generated tests and LLMs' test generation.

A number of LLM-based test generation techniques have been proposed [40]. CodaMosa [21] addresses the coverage plateau by prompting an LLM for a new test case when the search-based test generation process stalls. Bareiß *et al.* [4] evaluate the performance of Codex on Java unit test generation. Their prompts include the method signature, an example test case, and the method body, while discarding any non-compiling outputs. Vikram *et al.* [39] investigate the use of modern LLMs to automatically synthesize property-based tests using API documentation. TiCoder [15, 19] prompts LLMs to generate tests from textual descriptions of intended code functionality, enabling test-driven development. TestPilot [35] generates JavaScript unit tests by prompting an LLM with the function under test, its documentation, and related usage examples. ChatUniTest [8] focuses on Java unit test generation by prompting LLMs with source code; it also includes mechanisms to repair non-compiling tests. TestEval [41] benchmarks LLMs on LeetCode problems, while TestGenEval [18] is a project-level suite focused on testing goals like exception handling and boundary coverage.

Fuzz4All [42] uses two LLMs in tandem to perform fuzz testing across programming languages. SymPrompt [34] aims to generate tests that reach hard-to-cover code regions via prompting LLMs to focus on their path constraints. TestGen-LLM [2] prompts the LLM with the class under test—and optionally the existing test class—to produce additional coverage-enhancing test cases. MuTAP [11] focuses on mutation testing in Python. It prompts the LLM to generate an initial test, performs mutation testing, and prompts again to generate new assertions for surviving mutants, which are added to the test suite. ExLong [45] is an instruction fine-tuned LLM built on CodeLlama to reason on traces to methods containing throw statements or guard expressions for exceptional behavior tests.

Several approaches assist the LLMs in reasoning about the dynamic program behaviors [5, 9, 13, 14, 22, 24, 25, 27, 28, 30, 33, 38]. IPA-GNN [5] is a GNN variant tailored toward leverage CFGs to predict program execution. VisualCoder [9] leverages multi-modal Chain-of-Thought (CoT) reasoning to improve the execution prediction using CFG expressed in image and text representation. PredEx [22] combines LLMs with predictive backward slicing to predict the next executed statement. However, IPA-GNN, VisualCoder, and PredEx predict program execution traces and variable values from a given input without actual execution. In contrast, TestWeaver generates an input test case to cover a target statement.

La Malfa *et al.* [28] propose a benchmark to assess LLMs in simulating complex code. Their Chain of Simulation instructs LLMs to simulate program execution following the computation pattern of compilers. Similarly, Orca [33] leverages ReAct planning [43] to instruct LLMs to predict program execution to detect runtime errors. CodePilot [13] utilizes CoT planning to predict code coverage for a given input, improving over simple prompting in Tufano *et al.* [38]. Liu *et al.* [25] propose to pre-train LLMs with execution data. Lyu *et al.* [27] introduce Iterative Instruction Prompting (IIP) to improve over CoT in code execution prediction. TRACED [14] is an execution-aware pre-training strategy for source code with a combination of source code, executable inputs, and execution traces. These planning/prompting frameworks and execution-aware pre-train strategies can be used to enhance our prompting in the test case re-generation phase because they can help the LLM learn the execution nuances. Inspired by NeXT [30], we used execution inlining to provide focused context for execution reasoning. CodeMind [24] aims to gauge LLMs' reasoning via inductive reasoning. Deep learning has been used to infer inputs from desired outputs via program synthesis, input-output reasoning [7, 12, 29, 32, 44].

11 Conclusion and Implications

Novelty. This paper introduced TestWeaver, a novel LLM-based framework that overcomes the limitations of existing test generation approaches by integrating execution-aware constructive feedback into the test generation loop. Unlike prior work that relies solely on passive retrying in prompting, TestWeaver brings three key innovations: backward slicing to focus the model's attention on execution-relevant code, retrieval of control-flow-"closest" test cases to provide context-efficient guidance, and execution in-line annotations to help LLMs reason more effectively about program behavior. These techniques jointly address the issue of coverage plateau by equipping LLMs with targeted, execution-aware inputs that reduce redundancy and improve path exploration.

Implications. TestWeaver opens new avenues for future research at the intersection of program analysis and large language models in software testing. By demonstrating that lightweight static and dynamic analyses can meaningfully augment LLM reasoning, TestWeaver encourages a shift from purely prompt-based techniques to hybrid methods that integrate symbolic information and execution context. This approach could inspire the development of smarter, context-aware LLM agents not only for regression testing but also for broader tasks such as debugging, program repair, and test oracle inference. Furthermore, TestWeaver's notion of "closeness" between execution paths may serve as a foundation for designing novel guidance strategies in other test generation paradigms, such as symbolic or concolic testing. In practical settings, TestWeaver's ability to generate high-coverage test cases could significantly reduce bugs and accelerate continuous integration.

Data Availability. https://github.com/FSoft-AI4Code/TestWeaver

Acknowledgments

This work was supported in part by the US National Science Foundation (NSF) grant CNS-2120386.

References

[1] Hiralal Agrawal and Joseph R. Horgan. 1990. Dynamic program slicing. In *Proceedings of the ACM SIGPLAN 1990 Conference on Programming Language Design and Implementation* (White Plains, New York, USA) *(PLDI '90)*. Association for Computing Machinery, New York, NY, USA, 246–256. doi:10.1145/93542.93576

[2] Nadia Alshahwan, Jubin Chheda, Anastasia Finogenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. 2024. Automated Unit Test Improvement using Large Language Models at Meta. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering* (Porto de Galinhas, Brazil) *(FSE 2024)*. Association for Computing Machinery, New York, NY, USA, 185–196. doi:10.1145/3663529.3663839

[3] Juan Altmayer Pizzorno and Emery D. Berger. 2025. CoverUp: Effective High Coverage Test Generation for Python. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE128 (June 2025), 23 pages. doi:10.1145/3729398

[4] Patrick Bareiß, Beatriz Souza, Marcelo d'Amorim, and Michael Pradel. 2022. Code Generation Tools (Almost) for Free? A Study of Few-Shot, Pre-Trained Language Models on Code. arXiv:2206.01335 [cs.SE] https://arxiv.org/abs/2206.01335

[5] David Bieber, Charles Sutton, Hugo Larochelle, and Daniel Tarlow. 2019. Learning to execute programs with instruction pointer attention graph neural networks. *Advances in Neural Information Processing Systems* 33 (2019), 8626–8637.

[6] Junkai Chen, Zhiyuan Pan, Xing Hu, Zhenhao Li, Ge Li, and Xin Xia. 2025. Reasoning Runtime Behavior of a Program with LLM: How Far are We? . In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Los Alamitos, CA, USA, 1869–1881. doi:10.1109/ICSE55347.2025.00012

[7] Xinyun Chen, Dawn Song, and Yuandong Tian. 2021. Latent execution for neural program synthesis beyond domain-specific languages. *Advances in Neural Information Processing Systems* 34 (2021), 22196–22208.

[8] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. ChatUniTest: A Framework for LLM-Based Test Generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering* (Porto de Galinhas, Brazil) *(FSE 2024)*. Association for Computing Machinery, New York, NY, USA, 572–576. doi:10.1145/3663529.3663801

[9] Cuong Le Chi, Chau Truong Vinh Hoang, Phan Nhat Huy, Dung D. Le, Tien N Nguyen, and Nghi D. Q. Bui. 2025. VisualCoder: Guiding Large Language Models in Code Execution with Fine-grained Multimodal Chain-of-Thought Reasoning. In *Findings of the Association for Computational Linguistics: NAACL 2025*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 6628–6645. doi:10.18653/v1/2025.findings-naacl.370

[10] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. 2024. Effective test generation using pre-trained Large Language Models and mutation testing. *Inf. Softw. Technol.* 171, C (July 2024), 17 pages. doi:10.1016/j.infsof.2024.107468

[11] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. 2024. Effective test generation using pre-trained Large Language Models and mutation testing. *Information and Software Technology* 171 (2024), 107468. doi:10.1016/j.infsof.2024.107468

[12] Jacob Devlin, Jonathan Uesato, Surya Bhupatiraju, Rishabh Singh, Abdel-rahman Mohamed, and Pushmeet Kohli. 2017. RobustFill: neural program learning under noisy I/O. In *Proceedings of the 34th International Conference on Machine Learning - Volume 70* (Sydney, NSW, Australia) *(ICML'17)*. JMLR.org, 990–998.

[13] Hridya Dhulipala, Aashish Yadavally, and Tien N. Nguyen. 2024. Planning to Guide LLM for Code Coverage Prediction. In *Proceedings of the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering* (Lisbon, Portugal) *(FORGE '24)*. Association for Computing Machinery, New York, NY, USA, 24–34. doi:10.1145/3650105.3652292

[14] Yangruibo Ding, Benjamin Steenhoek, Kexin Pei, Gail Kaiser, Wei Le, and Baishakhi Ray. 2024. TRACED: Execution-aware Pre-training for Source Code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 36, 12 pages. doi:10.1145/3597503.3608140

[15] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shuvendu K. Lahiri. 2024. LLM-Based Test-Driven Interactive Code Generation: User Study and Empirical Evaluation. *IEEE Transactions on Software Engineering* 50, 9 (2024), 2254–2268. doi:10.1109/TSE.2024.3428972

[16] Patrice Godefroid, Nils Klarlund, and Koushik Sen. 2005. DART: directed automated random testing. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation* (Chicago, IL, USA) *(PLDI '05)*. Association for Computing Machinery, New York, NY, USA, 213–223. doi:10.1145/1065010.1065036

[17] Adrian Herrera, Hendra Gunadi, Shane Magrath, Michael Norrish, Mathias Payer, and Antony L. Hosking. 2021. Seed selection for successful fuzzing. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis* (Virtual, Denmark) *(ISSTA 2021)*. Association for Computing Machinery, New York, NY, USA, 230–243. doi:10.1145/3460319.3464795

[18] Kush Jain, Gabriel Synnaeve, and Baptiste Roziere. 2025. TestGenEval: A Real World Unit Test Generation and Test Completion Benchmark. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=7o6SG5gVev

[19] Shuvendu K. Lahiri, Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, Madanlal Musuvathi, Piali Choudhury, Curtis von Veh, Jeevana Priya Inala, Chenglong Wang, and Jianfeng Gao. 2023. Interactive Code Generation via Test-Driven User-Intent Formalization. arXiv:2208.05950 [cs.SE] https://arxiv.org/abs/2208.05950

[20] Cuong Chi Le, Hoang Nhat Phan, Huy Nhat Phan, Tien N. Nguyen, and Nghi D. Q. Bui. 2025. CodeFlow: Program Behavior Prediction with Dynamic Dependencies Learning . In *Proceedings of 2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*. IEEE Computer Society, Los Alamitos, CA, USA, 192–203. doi:10.1109/Forge66646.2025.00029

[21] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-Trained Large Language Models. In *Proceedings of the 45th International Conference on Software Engineering* (Melbourne, Victoria, Australia) *(ICSE '23)*. IEEE Press, 919–931. doi:10.1109/ICSE48619.2023.00085

[22] Yi Li, Hridya Dhulipala, Aashish Yadavally, Xiaokai Rong, Shaohua Wang, and Tien N. Nguyen. 2025. Blended Analysis for Predictive Execution. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE132 (June 2025), 22 pages. doi:10.1145/3729402

[23] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).

[24] Changshu Liu, Yang Chen, and Reyhaneh Jabbarvand. 2025. CodeMind: Evaluating Large Language Models for Code Reasoning. arXiv:2402.09664 [cs.SE] https://arxiv.org/abs/2402.09664

[25] Chenxiao Liu, Shuai Lu, Weizhu Chen, Daxin Jiang, Alexey Svyatkovskiy, Shengyu Fu, Neel Sundaresan, and Nan Duan. 2023. Code Execution with Pre-trained Language Models. In *Findings of the Association for Computational Linguistics: ACL 2023*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 4984–4999. doi:10.18653/v1/2023.findings-acl.308

[26] Stephan Lukasczyk, Florian Kroiß, and Gordon Fraser. 2023. An empirical study of automated unit test generation for Python. *Empirical Softw. Engg.* 28, 2 (Jan. 2023), 46 pages. doi:10.1007/s10664-022-10248-w

[27] Chenyang Lyu, Lecheng Yan, Rui Xing, Wenxi Li, Younes Samih, Tianbo Ji, and Longyue Wang. 2024. Large Language Models as Code Executors: An Exploratory Study. arXiv:2410.06667 [cs.CL] https://arxiv.org/abs/2410.06667

[28] Emanuele La Malfa, Christoph Weinhuber, Orazio Torre, Fangru Lin, Samuele Marro, Anthony Cohn, Nigel Shadbolt, and Michael Wooldridge. 2024. Code Simulation Challenges for Large Language Models. arXiv:2401.09074 [cs.LG] https://arxiv.org/abs/2401.09074

[29] Tural Mammadov, Dietrich Klakow, Alexander Koller, and Andreas Zeller. 2025. Learning Program Behavioral Models from Synthesized Input-Output Pairs. *ACM Trans. Softw. Eng. Methodol.* (July 2025). doi:10.1145/3748720

[30] Ansong Ni, Miltiadis Allamanis, Arman Cohan, Yinlin Deng, Kensen Shi, Charles Sutton, and Pengcheng Yin. 2024. NExT: Teaching Large Language Models to Reason about Code Execution. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*, Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (Eds.). PMLR, 37929–37956. https://proceedings.mlr.press/v235/ni24a.html

[31] Shankara Pailoor, Andrew Aday, and Suman Jana. 2018. Moonshine: optimizing OS fuzzer seed selection with trace distillation. In *Proceedings of the 27th USENIX Conference on Security Symposium* (Baltimore, MD, USA) *(SEC'18)*. USENIX Association, USA, 729–743.

[32] Emilio Parisotto, Abdel rahman Mohamed, Rishabh Singh, Lihong Li, Dengyong Zhou, and Pushmeet Kohli. 2017. Neuro-Symbolic Program Synthesis. In *International Conference on Learning Representations (ICLR'17)*. https://openreview.net/forum?id=rJ0JwFcex

[33] Smit Patel, Aashish Yadavally, Hridya Dhulipala, and Tien Nguyen. 2025. Planning a Large Language Model for Static Detection of Runtime Errors in Code Snippets . In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Los Alamitos, CA, USA, 639–639. doi:10.1109/ICSE55347.2025.00102

[34] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. 2024. Code-Aware Prompting: A Study of Coverage-Guided Test Generation in Regression Setting using LLM. *Proc. ACM Softw. Eng.* 1, FSE, Article 43 (July 2024), 21 pages. doi:10.1145/3643769

[35] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE Transactions on Software Engineering* 50, 1 (2024), 85–105. doi:10.1109/TSE.2023.3334955

[36] Koushik Sen, Darko Marinov, and Gul Agha. 2005. CUTE: a concolic unit testing engine for C. In *Proceedings of the 10th European Software Engineering Conference Held Jointly with 13th ACM SIGSOFT International Symposium on Foundations of*

Software Engineering (Lisbon, Portugal) *(ESEC/FSE-13)*. Association for Computing Machinery, New York, NY, USA, 263–272. doi:10.1145/1081706.1081750

[37] TestWeaver [n. d.]. TestWeaver. https://test-weaver.site.

[38] Michele Tufano, Shubham Chandel, Anisha Agarwal, Neel Sundaresan, and Colin Clement. 2023. Predicting Code Coverage without Execution. arXiv:2307.13383 [cs.SE]

[39] Vasudev Vikram, Caroline Lemieux, Joshua Sunshine, and Rohan Padhye. 2024. Can Large Language Models Write Good Property-Based Tests? arXiv:2307.04346 [cs.SE] https://arxiv.org/abs/2307.04346

[40] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software Testing With Large Language Models: Survey, Landscape, and Vision. *IEEE Transactions on Software Engineering* 50, 4 (2024), 911–936. doi:10.1109/TSE.2024.3368208

[41] Wenhan Wang, Chenyuan Yang, Zhijie Wang, Yuheng Huang, Zhaoyang Chu, Da Song, Lingming Zhang, An Ran Chen, and Lei Ma. 2025. TestEval: Benchmarking Large Language Models for Test Case Generation. In *Findings of the Association for Computational Linguistics: NAACL 2025*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 3547–3562. doi:10.18653/v1/2025.findings-naacl.197

[42] C. Xia, M. Paltenghi, J. Tian, M. Pradel, and L. Zhang. 2024. Fuzz4ALL: Universal Fuzzing with Large Language Models. In *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Los Alamitos, CA, USA, 1547–1559. https://doi.ieeecomputersociety.org/

[43] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the Eleventh International Conference on Learning Representations (ICLR)*.

[44] Wojciech Zaremba and Ilya Sutskever. 2015. Learning to Execute. arXiv:1410.4615 [cs.NE] https://arxiv.org/abs/1410.4615

[45] Jiyang Zhang, Yu Liu, Pengyu Nie, Junyi Jessy Li, and Milos Gligoric. 2025. exLong: Generating Exceptional Behavior Tests with Large Language Models. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. 1462–1474. doi:10.1109/ICSE55347.2025.00176